

AI Chat / Agent Orchestrator 架构方案

减法版实现：小模型路由 + 能力注册表 + RAG/Memory + 大模型交付

一句话定义

Orchestrator 不是一个模型，而是一套由代码主导的 AI 调度系统。小模型负责判断和抽取，Embedding 负责匹配和召回，代码负责决策和执行，大模型负责最终综合与表达。

文档范围

文档目标

抽象并定义一套减法版 AI Chat / Agent Orchestrator 架构，明确职责、链路、模型使用位置和边界。

适用产品

个人知识库、采集中心、项目资料问答、自动入库、轻量工作流、未来可扩展 Agent 的 AI 应用。

设计原则

先可控，再智能；先 Workflow，再 Agent；先筛选上下文，再交给大模型。

不包含内容

不输出具体代码、不讨论某个框架的 API 细节、不设计完整通用 Agent 平台。

1. 架构目标与核心边界

这套架构要解决的问题，不是“让一个模型变得无所不能”，而是让系统在每次用户请求进来时，能够稳定判断应该走哪条处理路径，并把必要信息组织成一个可控的任务包。

- 目标一：**识别请求类型，决定走 Chat、RAG Chat、Skill、Workflow 或未来的 Agent。
- 目标二：**减少大模型负担，让大模型只处理已经整理过的上下文，而不是直接面对原始资料堆。
- 目标三：**把工具调用、流程执行、上下文召回和最终生成分离，避免一个模型同时承担所有职责。
- 目标四：**建立清晰边界：小模型不做最终结论，Workflow 不做开放式推理，Agent 不处理固定流程。

关键边界

Orchestrator 的输出不是答案，而是 RoutePlan。RoutePlan 描述“这个请求应该如何处理”，最终答案由后续执行链路和大模型共同完成。

2. 总览架构

减法版 Orchestrator 保留最小但完整的骨架：小模型路由、能力注册表匹配、RAG/Memory 召回、上下文构建、最终大模型生成。

层级	核心职责	是否使用模型	边界
输入层	接收用户请求、当前会话、附件、项目上下文	否	不做判断，只收集请求材料
Router 层	判断任务类型、上下文需求、能力需求、抽取关键参数	是，小模型	只输出结构化判断，不回答用户
能力匹配层	将非 Chat 请求匹配到 Skill	是，Embedding 为主	不直接执行高风险动作

	或 Workflow		
召回层	从文档、项目、历史摘要、Memory 中找相关信息	是，Embedding/Reranker/小模型	只召回和排序，不生成最终答案
执行层	执行 Skill 或 Workflow，得到可记录结果	视任务而定	固定流程优先，不让 Agent 乱接管
上下文构建层	筛选、压缩、组装最终上下文包	代码为主，模型辅助	不是合并全部信息，而是合并必要信息
生成层	基于上下文输出最终答案或交付物	是，大模型	只基于输入材料，不重新决定系统路径

抽象链路

步骤	处理动作	输出
1	用户请求进入 Orchestrator	RequestContext
2	小模型 Router 判断请求类型和需求	RoutePlan
3A	如果是 Chat/RAG Chat，进入召回规划与上下文召回	RetrievedContext
3B	如果是 Skill/Workflow，进入能力注册表匹配与参数检查	CapabilityPlan
4	执行必要能力或整理召回结果	ExecutionResult / EvidencePack
5	Context Builder 组装最终任务包	FinalContext
6	大模型生成最终结果	Answer / Report / Summary

3. 核心对象：RequestContext、RoutePlan、FinalContext

为了让系统可观测、可回放、可调试，Orchestrator 内部不应该在模块之间传散乱字符串，而应该传递稳定的结构化对象。

对象	含义	主要内容	生命周期
RequestContext	用户请求进入系统后的标准化上下文	用户输入、当前会话、项目、附件、用户身份、环境信息	进入 Router 前生成
RoutePlan	Orchestrator 的核心判断结果	任务类型、是否需要 RAG/Memory/Skill/Workflow、候选能力、参数、置信度	Router 输出，后续链路依据它执行
CapabilityPlan	非 Chat 请求的能力执行计划	匹配到的 Skill/Workflow、参数、风险、成本、缺失项	能力匹配后生成
EvidencePack	召回后经过筛选的证据包	文档片段、Memory、历史摘要、来源、相关度、压缩摘要	RAG/Memory 路径生成
ExecutionResult	Skill/Workflow 执行结果	执行状态、产物、日志、错误、入库位置、摘要	能力执行后生成
FinalContext	交给大模型的最终上下文包	用户目标、任务类型、证据、执行结果、约束、输出格式	最终生成前生成

设计边界

RoutePlan 是系统决策的中间产物，不直接展示给用户。它应该可日志化、可回放、可用于评估路由准确率。

4. Router 层：小模型做什么，不做什么

Router 是减法版 Orchestrator 的第一判断点。它可以直接使用一个小模型完成，不需要多个小模型投票。更稳妥的方式是：一个小模型负责结构化判断，Embedding 负责后续匹配，代码负责最终裁决。

Router 任务	说明	模型选择	输出
任务类型判断	判断请求属于 chat、rag_chat、skill_call、workflow，未来可扩展 agent	小模型	task_type
上下文需求判断	判断是否需要 memory、history summary、project docs、current conversation	小模型	needs_rag / needs_memory
能力需求判断	判断是否需要调用 Skill 或 Workflow	小模型	needs_skill / needs_workflow
参数抽取	从用户请求中提取项目名、文件类型、URL、目标动作等	小模型	extracted_params
能力匹配查询词生成	为 Skill/Workflow Registry 匹配生成语义查询	小模型	capability_query
RAG 检索词生成	为文档和 Memory 召回生成搜索 query	小模型	search_queries
缺失参数判断	判断当前请求是否缺少执行所需参数	小模型 + 代码校验	missing_params
置信度输出	给出结构化判断的可信程度	小模型	confidence

Router 不应该承担的职责

- 不直接回答用户问题。
- 不直接执行工具或工作流。
- 不直接决定高风险动作是否可以执行。
- 不做复杂产品分析、商业判断、长文档综合结论。
- 不负责最终输出质量，只负责把请求分到正确路径。

5. 任务类型与路径边界

为了降低复杂度，第一版只需要四类路径：Chat、RAG Chat、Skill Call、Workflow。Agent 可以作为后续扩展，不进入 MVP 的默认路径。

路径	适用情况	不适用情况	后续动作
Chat	解释、分析、写作、改写、普通问答，不依赖用户资料	需要查项目资料、执行动作、处理文件	直接进入 Final LLM，必要时仅携带当前会话

RAG Chat	需要基于文档、历史摘要、Memory、项目资料回答	需要改变系统状态或执行固定流程	召回、重排、压缩、构建证据包，再交给大模型
Skill Call	一个单一能力即可完成，例如搜索笔记、生成单篇摘要、创建简单记录	需要多步骤固定流程或开放式规划	匹配 Skill、校验参数、执行、返回结果或交给大模型整理
Workflow	固定流程任务，例如 URL 入库、PDF 解析、音视频转写、切片向量化	路径不确定、需要自主研究或多轮策略判断	匹配 Workflow、检查参数、执行步骤、记录状态
Agent	目标明确但路径开放，需要多步计划、工具调用、自检和补充行动	MVP 默认不启用；固定流程不应走 Agent	后续作为独立 Runtime 接入

最重要的分界线

Workflow 是“预设路径的执行”；Agent 是“动态决定下一步”。凡是路径固定的任务，都不要交给 Agent。

6. 能力注册表：Skill Registry 与 Workflow Registry

非 Chat 请求不应该让小模型凭空决定调用什么能力，而是先进入能力注册表匹配。注册表是系统能力的“菜单”，也是 Orchestrator 可控性的根。

注册项	Skill Registry	Workflow Registry
定义	单一能力，通常一步或少量步骤完成	固定流程，由多个确定步骤组成
典型能力	语义搜索、生成摘要、创建笔记、提取关键词	URL 入库、PDF 解析、音频转写、视频入库、批量导入
匹配方式	用户请求与能力描述、触发样例做语义匹配	用户请求与流程描述、触发样例做语义匹配
必要字段	名称、描述、输入参数、输出格式、风险、成本、是否确认	名称、描述、步骤、输入参数、输出产物、失败处理、风险、成本
执行边界	不能跨越自身能力范围	必须按预定义步骤执行，不做开放式推理
是否需要大模型	视能力而定，部分 Skill 可直接返回	通常只在摘要、解释、最终交付时需要大模型

能力匹配原则

- 先用小模型生成 capability_query，再用 Embedding 匹配注册表。
- 匹配结果只作为候选，不等于可以执行。
- 执行前必须校验 required params 是否完整。
- 涉及删除、发布、发送、付款、外部写入等动作时，必须增加人工确认。
- 当 Skill 与 Workflow 都匹配时，优先选择边界更明确、步骤更确定的路径。

7. RAG 与 Memory：召回不是默认全开

Chat 路径不调用 Skill/Workflow，但也不意味着每次都查全部 RAG 和 Memory。召回应该由 Router 的上下文需求判断驱动。

召回源	适用条件	边界
当前会话	问题依赖刚刚的上下文、追问、引用“上面/刚才/这个”	不要无限携带完整历史，会话过长时需要摘要
用户 Memory	问题依赖用户长期偏好、长期项目背景、稳定身份信息	不要把所有个人信息默认加入上下文
项目 Memory	问题与某个项目的定位、约束、历史决策有关	必须绑定 project_id 或可明确识别的项目
历史摘要	问题涉及过去讨论但不要求精确原文	摘要可能丢细节，关键事实需要再查原始记录
文档 RAG	问题需要基于上传文档、网页、笔记、音视频转写内容回答	召回片段必须有来源，避免无证据生成
工具结果	Workflow/Skill 已经执行并产生结果	工具结果优先级高于模型推测

召回边界

RAG 和 Memory 是证据来源，不是答案本身。最终模型必须知道哪些内容来自证据，哪些是推理，哪些是不确定。

8. 上下文构建：从“合并全部信息”改为“构建任务包”

最终交给大模型的不是所有材料，而是经过筛选、压缩、排序后的 FinalContext。Context Builder 是保证质量和成本的核心。

步骤	动作	目的
筛选	剔除低相关、重复、过期、冲突未标注的信息	避免模型被噪音带偏
排序	按相关性、时效性、来源可信度、任务必要性排序	让重要信息靠前
压缩	将长文档片段压成证据摘要，保留关键原文与来源	控制 token 成本
分组	按用户目标、项目背景、证据、执行结果、约束分组	降低大模型理解成本
标注	标明来源、时间、置信度、是否为推测	方便最终回答更稳
预算控制	为系统提示、会话、Memory、RAG、工具结果分别设 token 上限	防止上下文爆炸

FinalContext 应包含

- 用户原始请求与标准化目标。
- 任务类型和处理路径。
- 必要的当前会话。
- 筛选后的 Memory 与项目背景。
- RAG 证据包及来源。
- Skill/Workflow 执行结果。
- 输出要求、格式要求、禁止事项。
- 不确定点与缺失信息。

9. 模型使用分工

减法版架构不需要一堆小模型并行判断。推荐使用“一个小模型做 Router + Embedding 做匹配/召回 + 可选 Reranker + 大模型最终生成”的组合。

模型/组件	使用位置	负责事项	不负责事项
小模型 Router	请求进入后第一层判断	任务分类、需求判断、参数抽取、检索词生成	最终答案、复杂规划、高风险决策
Embedding 模型	Skill/Workflow 匹配，RAG/Memory 召回	语义相似度匹配、候选召回	理解复杂业务结论
Reranker	召回后排序	判断候选片段与问题的相关性	生成答案
压缩模型	长证据处理时可选	压缩、去重、提炼证据	替代最终大模型判断
大模型	最终回答或最终交付	综合推理、表达、报告生成、复杂解释	直接控制系统路径、未经约束调用能力
代码规则	贯穿整个 Orchestrator	状态管理、参数校验、风险控制、执行路由	自然语言理解与复杂生成

模型边界

小模型给判断，Embedding 给候选，代码给裁决，大模型给最终表达。不要让任何一个模型独自掌权。

10. 执行控制与风险边界

Orchestrator 必须控制执行，而不是让模型自由调用能力。每个路径都需要参数校验、风险等级、失败处理和可观测日志。

控制点	要求	原因
参数完整性	required params 缺失时不得执行	避免执行到一半失败或写错项目
风险等级	低风险可自动执行，高风险需要确认	防止误删、误发、误发布、误付款
成本等级	高成本流程需要预算控制或确认	避免音视频、大模型、多次调用成本失控
状态记录	Workflow 每一步需要记录状态和产物	便于失败恢复、用户追踪、日志审计
错误处理	失败要返回明确原因和可恢复路径	不能用含糊话术掩盖失败
幂等性	入库、导入、转写等动作需要避免重复写入	防止用户资料污染
人工确认	外部写入、高风险动作、不可逆动作必须确认	保护用户和系统

11. Agent 的接入边界

这套减法版架构可以先不开放完整 Agent。Agent 应该作为后续独立 Runtime 接入，而不是混在 Workflow 或 Chat 里。

问题	边界
什么时候需要 Agent	当用户给的是目标，而不是固定动作；路径不确定，需要多

	步计划、工具调用、自检和补充行动。
什么时候不需要 Agent	文件解析、音频转写、URL 入库、PDF 切片、单文档摘要、项目内问答。
Agent 必须有什么	最大步骤数、最大工具调用次数、预算、允许工具清单、停止条件、状态日志、人工确认点。
Agent 不应该做什么	不接管所有请求，不处理固定流程，不跳过证据验证，不在无确认情况下执行高风险动作。
如何接入现有架构	Router 未来增加 agent 类型；Agent 执行结束后把结果作为 ExecutionResult 进入 FinalContext。

Agent 边界

Agent 是开放式任务执行器，不是“更聪明的 Chat”。没有状态、预算、停止条件和工具边界的 Agent，不应进入生产系统。

12. 推荐 MVP 形态

第一版不要追求完整 Agent 平台，而是先把 Orchestrator 的基础路径跑稳。

阶段	能力范围	目标
MVP 1	小模型 Router、RAG Chat、Memory 召回、Final LLM	让项目内问答、资料总结、历史上下文使用稳定
MVP 2	Skill/Workflow Registry、URL/PDF/音视频入库流程	让采集、解析、入库形成闭环
MVP 3	Context Builder、证据压缩、召回重排、token 预算	降低成本，提高回答稳定性
MVP 4	少量轻 Agent：项目资料整理、竞品分析、深度报告	只在高价值场景开放开放式执行
MVP 5	定时任务、监控、长期执行状态管理	从一次性任务扩展到持续任务

13. 判断与验收标准

这套架构是否有效，不能只看 demo 是否“看起来聪明”，而要看稳定性、成本、边界和可解释性。

指标	验收标准
路由准确率	Chat、RAG Chat、Skill、Workflow 的分类错误率可被统计，并能通过样本集持续优化。
召回质量	最终答案依赖的关键证据能被召回，且无关证据不过量进入上下文。
上下文成本	每类请求有明确 token 预算，长文档场景不会无限膨胀。
执行成功率	Workflow 的每一步有状态记录，失败原因可追踪。
边界安全	高风险动作不会被模型直接执行，必须经过确认或策略检查。
可调试性	每次请求都能回放 RoutePlan、候选能力、召回证据、FinalContext。
用户体验	简单问题不被复杂化，复杂任务能给出明确进度、结果和缺失项。

14. 最终架构定义

最终定义

这套 Orchestrator 是一个“代码主导、模型辅助”的请求调度系统：小模型负责判断请求去向，Embedding 负责找到相关能力和资料，代码负责执行与边界控制，大模型负责基于整理后的上下文完成最终交付。

它的价值不在于让系统显得更像 Agent，而在于让每类请求都走最合适、最低成本、最可控的路径。

- 能直接回答的，不调用能力。
- 需要资料的，走 RAG / Memory。
- 流程固定的，走 Workflow。
- 单一动作的，走 Skill。
- 路径开放且高价值的，才进入 Agent。
- 最终大模型只接收整理后的任务包，而不是原始信息垃圾场。

一句话收束

Orchestrator 的职责不是“变聪明”，而是“让聪明被约束、被路由、被记录、被验证”。这是 AI 产品从玩具走向系统的分界线。